

Python Code

```

1  """
2  QR Code Generator
3  """
4
5  import comp140_module5 as qrcode
6  import comp140_module5_z256 as z256
7
8  def divide_terms(coefficient1, power1, coefficient2, power2):
9      """
10         Computes the quotient of two terms.
11
12         The degree of the first term, power1, must be greater than or
13         equal to the degree of the second term, power2.
14
15         Inputs:
16             - coefficient1: a Z_256 number representing the coefficient of the first
17               polynomial term
18             - power1: an integer representing the power of the first term.
19             - coefficient2: a Z_256 number representing the coefficient of the second
20               polynomial term
21             - power2: an integer representing the power of the second term.
22
23         Returns: an instance of a Polynomial that is the resulting
24               term.
25         """
26         # From recipe: (a*x^b) / (c*x^d) = (a/c) * x^(b-d)
27         new_coeff = z256.div(coefficient1, coefficient2)
28         new_pow = power1 - power2
29
30         # Represent our answer as a Polynomial
31         divided = Polynomial()
32         divided = divided.add_term(new_coeff, new_pow)
33         return divided
34
35  class Polynomial:
36      """
37         A class used to abstract methods on a polynomial in the finite
38         field Z_256 (including numbers from 0 through 255).
39
40         Since 256 is not prime, but is rather of the form p^n = 2^8, this
41         representation uses special arithmetic via the z256 module so as to
42         preserve multiplicative inverses (division) inside this field.
43         """
44
45         def __init__(self, terms=None):
46             """
47                 Creates a new Polynomial object. If a dictionary of terms is provided,
48                 they will be the terms of the polynomial,
49                 otherwise the polynomial will be the 0 polynomial.

```

```

49         inputs:
50             - terms: a dictionary of terms mapping powers to coefficients or None
51               (None indicates that all coefficients are 0)
52         """
53         if terms != None:
54             self._terms = dict(terms)
55         else:
56             self._terms = {}
57
58     def __str__(self):
59         """
60         Returns: a string representation of the polynomial, containing the
61         class name and all of the terms.
62         """
63         # Create a string of the form "ax^n + bx^n-1 + ... + c" by
64         # creating a string representation of each term, and inserting
65         # " + " in between each
66         term_strings = []
67
68         # Add the highest powers first
69         powers = list(self._terms.keys())
70         powers.sort(reverse=True)
71         for power in powers:
72             coefficient = self._terms[power]
73             # Don't print out terms with a zero coefficient
74             if coefficient != 0:
75                 # Don't print "x^0"; that just means it's a constant
76                 if power == 0:
77                     term_strings.append("%d" % coefficient)
78                 else:
79                     term_strings.append("%d*x^%d" % (coefficient, power))
80
81         terms_str = " + ".join(term_strings)
82         if terms_str == "":
83             terms_str = "0"
84         return "Polynomial: %s" % terms_str
85
86     def __eq__(self, other_polynomial):
87         """
88         Check if another polynomial is equivalent
89
90         inputs:
91             - other_polynomial: a Polynomial object
92
93         Returns a boolean: True if other_polynomial contains
94         the same terms as self, False otherwise.
95         """
96         # Make sure that other_polynomial is a Polynomial
97         if not isinstance(other_polynomial, Polynomial):
98             return False
99
100        # Get the terms of the other_polynomial
101        terms = other_polynomial.get_terms()
102

```

```
103         # Check that all terms in other_polynomial appear in self
104         for power, coefficient in terms.items():
105             if coefficient != 0:
106                 if power not in self._terms:
107                     return False
108                 if self._terms[power] != coefficient:
109                     return False
110
111         # Check that all terms in self appear in other_polynomial
112         for power, coefficient in self._terms.items():
113             if coefficient != 0:
114                 if power not in terms:
115                     return False
116                 if terms[power] != coefficient:
117                     return False
118
119         return True
120
121     def __ne__(self, other_polynomial):
122         """
123         Check if another polynomial is NOT equivalent
124
125         inputs:
126             - other_polynomial: a Polynomial object
127
128         Return a boolean: False if other_polynomial contains the same terms
129         as self, True otherwise.
130         """
131         return not self.__eq__(other_polynomial)
132
133     def get_terms(self):
134         """
135         Returns: a dictionary of terms, mapping powers to coefficients.
136         This dictionary is a completely new object and is not a reference
137         to any internal structures.
138         """
139         terms = dict(self._terms)
140         return terms
141
142     def get_degree(self):
143         """
144         Returns: the maximum power over all terms in this polynomial.
145         """
146         # Since we don't clean zero-coefficient powers out of our dictionary,
147         # we need a trickier get_degree function, to take into account that
148         # some coefficients could be zero.
149         highest_power = 0
150         for power in self._terms:
151             if (power > highest_power) and (self._terms[power] != 0):
152                 highest_power = power
153
154         return highest_power
155
156
```

```
157 def get_coefficient(self, power):
158     """
159     Determines the coefficient of x^(power) in this polynomial.
160     If there is no coefficient of x^(power), this method
161     returns 0.
162
163     inputs:
164         - power: an integer representing a polynomial power
165
166     Returns: a Z_256 number that is the coefficient or 0 if there
167             is no term of the given power
168     """
169     if power in self._terms:
170         return self._terms[power]
171     else:
172         return 0
173
174 def add_term(self, coefficient, power):
175     """
176     Add one term to this polynomial.
177
178     inputs:
179         - coefficient: a Z_256 number representing the coefficient of the term
180         - power: an integer representing the power of the term
181
182     Returns: a new Polynomial that is the sum of adding this polynomial
183             to (coefficient) * x^(power) using Z_256 arithmetic to add
184             coefficients, if necessary.
185     """
186     # Replace with your code for part 3.A
187
188     #copy
189     result_terms = self.get_terms()
190     old_coeff = self.get_coefficient(power)
191     # add new term
192     result_terms[power] = z256.add(old_coeff, coefficient)
193     return Polynomial(result_terms)
194
195
196 def subtract_term(self, coefficient, power):
197     """
198     Subtract one term from this polynomial.
199
200     inputs:
201         - coefficient: a Z_256 number representing the coefficient of the term
202         - power: an integer representing the power of the term
203
204     Returns: a new Polynomial that is the difference of this polynomial
205             and (coefficient) * x^(power) using Z_256 arithmetic to subtract
206             coefficients, if necessary.
207     """
208     # deep copy for new poly
209     result_terms = self.get_terms()
210     old_coeff = self.get_coefficient(power)
```

```
211         # subtract
212         result_terms[power] = z256.sub(old_coeff, coefficient)
213         return Polynomial(result_terms)
214
215     def multiply_by_term(self, coefficient, power):
216         """
217         Multiply this polynomial by one term.
218
219         inputs:
220             - coefficient: a Z_256 number representing the coefficient of the term
221             - power: an integer representing the power of the term
222
223         Returns: a new Polynomial that is the product of multiplying
224         this polynomial by (coefficient) * x^(power).
225         """
226         # Replace with your code for part 3.C
227
228         #deep copy
229         result_terms = {}
230
231         # multiply in z256
232         for old_power, old_coefficient in self.get_terms().items():
233             result_terms[old_power + power] = z256.mul(old_coefficient, coefficient)
234
235         return Polynomial(result_terms)
236
237
238     def add_polynomial(self, other_polynomial):
239         """
240         Compute the sum of the current polynomial other_polynomial.
241
242         inputs:
243             - other_polynomial: a Polynomial object
244
245         Returns: a new Polynomial that is the sum of both polynomials.
246         """
247         # Replace with your code for part 4.A
248         #initialize
249         result = Polynomial(self.get_terms())
250
251         # add each of the terms from other poly
252         for power, coefficient in other_polynomial.get_terms().items():
253             result = result.add_term(coefficient, power)
254
255         return result
256
257     def subtract_polynomial(self, other_polynomial):
258         """
259         Compute the difference of the current polynomial and other_polynomial.
260
261         inputs:
262             - other_polynomial: a Polynomial object
263
264         Returns: a new Polynomial that is the difference of both polynomials.
```

```
265     """
266     # Replace with your code for part 4.
267     #initialize
268     result = Polynomial(self.get_terms())
269
270     # subtract each of the terms
271     for power, coefficient in other_polynomial.get_terms().items():
272         result = result.subtract_term(coefficient, power)
273
274     return result
275
276 def multiply_by_polynomial(self, other_polynomial):
277     """
278     Compute the product of the current polynomial and other_polynomial.
279
280     inputs:
281         - other_polynomial: a Polynomial object
282
283     Returns: a new Polynomial that is the product of both polynomials.
284     """
285     # Replace with your code for part 4.C
286     #initialize
287     result = Polynomial()
288
289     #multiply
290     for power, coeff in other_polynomial.get_terms().items():
291         intermediate_product = self.multiply_by_term(coeff, power)
292         result = result.add_polynomial(intermediate_product)
293
294     #return
295     return result
296
297
298 def remainder(self, denominator):
299     """
300     Compute a new Polynomial that is the remainder after dividing this
301     polynomial by denominator.
302
303     Note: does *not* return the quotient; only the remainder!
304
305     inputs:
306         - denominator: a Polynomial object
307
308     Returns: a new polynomial that is the remainder
309     """
310     # Replace with your code for part 4.D
311     # initialize
312     remainder = Polynomial(self.get_terms())
313
314     #iterate until remainder has degree less than denominator
315     while (remainder != Polynomial() and
316            remainder.get_degree() >= denominator.get_degree()):
317
318         # first term of numerator
```

```

319         n_power = remainder.get_degree()
320         n_coeff = remainder.get_coefficient(n_power)
321
322         # first term of denominator
323         d_power = denominator.get_degree()
324         d_coeff = denominator.get_coefficient(d_power)
325
326         # find the product that will be subtracted
327         quotient_term = divide_terms(n_coeff, n_power, d_coeff, d_power)
328         subtract_poly = denominator.multiply_by_polynomial(quotient_term)
329         remainder = remainder.subtract_polynomial(subtract_poly)
330
331     return remainder
332
333
334 def create_message_polynomial(message, num_correction_bytes):
335     """
336     Creates the appropriate Polynomial to represent the
337     given message. Relies on the number of error correction
338     bytes (k). The message polynomial is of the form
339     message[i]*x^(n+k-i-1) for each number/byte in the message.
340
341     Inputs:
342         - message: a list of integers (each between 0-255) representing data
343         - num_correction_bytes: an integer representing the number of
344           error correction bytes to use
345
346     Returns: a Polynomial with the appropriate terms to represent the
347     message with the specified level of error correction.
348     """
349     # Replace with your code for part 5.A
350     # initialize
351     message_poly = Polynomial()
352     length = len(message)
353
354     # sum over to get message poly
355     for index in range(length):
356         message_poly = message_poly.add_term(
357             message[index], num_correction_bytes + length - index - 1)
358
359     return message_poly
360
361 def create_generator_polynomial(num_correction_bytes):
362     """
363     Generates a static generator Polynomial for error
364     correction, which is the product of (x-2^i) for all i in the
365     set {0, 1, ..., num_correction_bytes - 1}.
366
367     Inputs:
368         - num_correction_bytes: desired number of error correction bytes.
369           In the formula, this is represented as k.
370
371     Returns: generator Polynomial for generating Reed-Solomon encoding data.
372     """

```

```
373     # Replace with your code for part 5.B
374     #initialize 1
375     generator_poly = Polynomial({0: 1})
376
377     # iterate over each factor poly and multiply to form generator
378     for index in range(num_correction_bytes):
379         factor_poly = Polynomial()
380         factor_poly = factor_poly.add_term(1, 1)
381         factor_poly = factor_poly.add_term(z256.power(2,index), 0)
382         # get poly which will be part of the product
383         generator_poly = generator_poly.multiply_by_polynomial(factor_poly)
384
385     return generator_poly
386
387 def reed_solomon_correction(encoded_data, num_correction_bytes):
388     """
389     Corrects the encoded data using Reed-Solomon error correction
390
391     Inputs:
392         - encoded_data: a list of integers (each between 0-255)
393           representing an encoded QR message.
394         - num_correction_bytes: desired number of error correction bytes.
395
396     Returns: a polynomial that represents the Reed-Solomon error
397     correction code for the input data.
398     """
399     # Replace with your code for part 5.C
400     numerator = create_message_polynomial(encoded_data, num_correction_bytes)
401     denominator = create_generator_polynomial(num_correction_bytes)
402     return numerator.remainder(denominator)
403
404
405     # Uncomment the following line when you are ready to generate an
406     # actual QR code. To do so, you must enter a short message in the
407     # "info" text box and hit return (be sure to hit return!). You then
408     # must push the "Generate!" button. This will generate a QR code for
409     # you to view - try scanning it with your phone! If you would like to
410     # save your QR codes, you can use the "Image in a New Window" button
411     # to create a .png file that you can save by right clicking in your
412     # browser window.
413
414     qrcode.start(reed_solomon_correction)
415
```